Title: Reinforcement Learning Agents might be able to align large telescopes

Abstract: Some telescopes use mirrors to reflect light into a camera. For very large telescopes, such as those found in CTAO and VERITAS, these mirrors are segmented. These mirror segments must be positioned very precisely for the telescope to function properly. The exact positions of these mirrors are difficult to find, and the current process for finding the correct positions does not reach a theoretical best alignment. We investigate using a Reinforcement Learning algorithm known as Proximal Policy Optimization (PPO) to optimize a neural network that finds these proper positions.

==========

V#.#
Idea:
Other noteworthy parameters:
Results:

==========

For version 6.#, we experiment on one panel without using images. Each agent still has memory (it can see previous frames), but the observation is the true centroid locations instead of the image. The action space available to the agent is a 3d vector containing: (panel_id, rx, ry).

The purpose of these experiments is to investigate how much effect the hyperparameters of training have on the training curves. If there isn't much effect then we can proceed to multiple panels without worrying about specific hyperparameters so long as they don't have atrocious values.

========== EXPERIMENT 1: PLAYING AROUND ==========

I change a bunch of variables and see how the training curves change.

V6.0
Idea:batchsize=32, n_epochs=5, pi=[256, 256, 128], vf=[128,64]
Other noteworthy parameters: logs=PPO_3
Results: see tensorboard logs

V6.1

```
model = PPO(
    policy="MlpPolicy",
    env=env,
    device="cpu",
    policy_kwargs = dict(
        net_arch=dict(
            pi=[32, 32],     # policy MLP
            vf=[32, 32]            # value MLP
        ),
        activation_fn=nn.ReLU,
    ),
    learning_rate=5e-5,
    n_steps=512,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.1,
    ent_coef=0.002,
    vf_coef=0.5,
    max_grad_norm=0.5,
    verbose=1,
    normalize_advantage=True,
    tensorboard_log="./ppo_logs/",
)
```

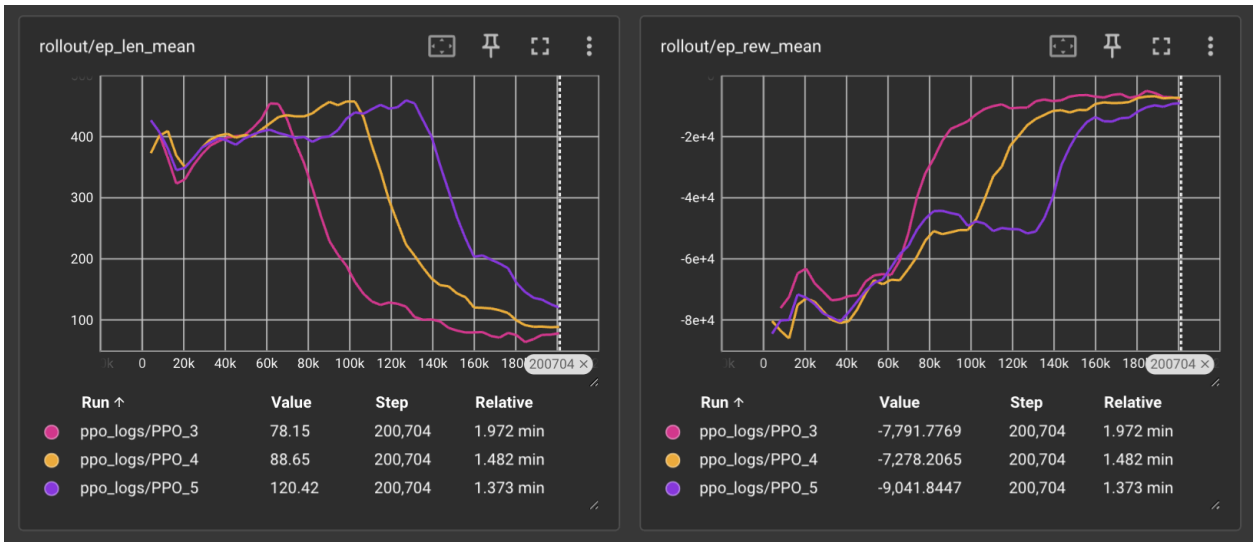
Idea:
Other noteworthy parameters: logs=PPO_4
Results: see tensorboard logs

V6.2

```
model = PPO(
    policy="MlpPolicy",
    env=env,
    device="cpu",
    policy_kwargs = dict(
        net_arch=dict(
            pi=[32, 32],     # policy MLP
            vf=[32, 32]            # value MLP
        ),
        activation_fn=nn.ReLU,
    ),
    learning_rate=5e-5,
    n_steps=512,
    batch_size=32,
    n_epochs=5,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.1,
    ent_coef=0.002,
    vf_coef=0.5,
    max_grad_norm=0.5,
    verbose=1,
    normalize_advantage=True,
    tensorboard_log="./ppo_logs/",
)
```

Idea:
Other noteworthy parameters: logs=PPO_5
Results: see tensorboard logs

V6.0-6.2 tensorboard logs:



Results: no results really to be had. I changed too many independent variables at the same time while conducting these experiments. But I now have a framework to rigorously test each architecture decision provided to me by the SB3 PPO algorithm.

==========EXPERIMENT 2: ARCHITECTURE ==========

experiment: see how each hyperparameter affects the performance of the agent. change one at a time. the default values will be:

```
model = PPO(
    policy="MlpPolicy",
    env=env,
    device="cpu",
    policy_kwargs = dict(
        net_arch=dict(
            pi=[128, 128, 128],         # policy MLP
            vf=[128, 128, 128]          # value MLP
        ),
        activation_fn=nn.ReLU,
    ),
    learning_rate=1e-4,
    n_steps=1024,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.1,
    ent_coef=0.002,
    vf_coef=0.5,
    max_grad_norm=0.5,
    verbose=1,
    normalize_advantage=True,
    tensorboard_log="./ppo_logs/lr_exp/",
)
```

also note that the amount of memory the agent receives is by default 3. This will start to matter more once we add more panels so we will wait to experiment with this number. All models will be trained for 200_000 timesteps

========== architecture experiment ==========

we set vf and pi to the following values:
[64, 64], [64, 64, 64], [64, 64, 64, 64], [128, 128], [128, 128, 128], [128, 128, 128, 128]

V6.3
Idea: [64, 64]
Other noteworthy parameters: logs=PPO_6
Results: see tensorboard logs

V6.4
Idea: [64, 64, 64]
Other noteworthy parameters: logs=PPO_7
Results: see tensorboard logs

V6.5
Idea: [64, 64, 64, 64]
Other noteworthy parameters: logs=PPO_8
Results: see tensorboard logs

V6.6
Idea: [128, 128]
Other noteworthy parameters: logs=PPO_9
Results: see tensorboard logs

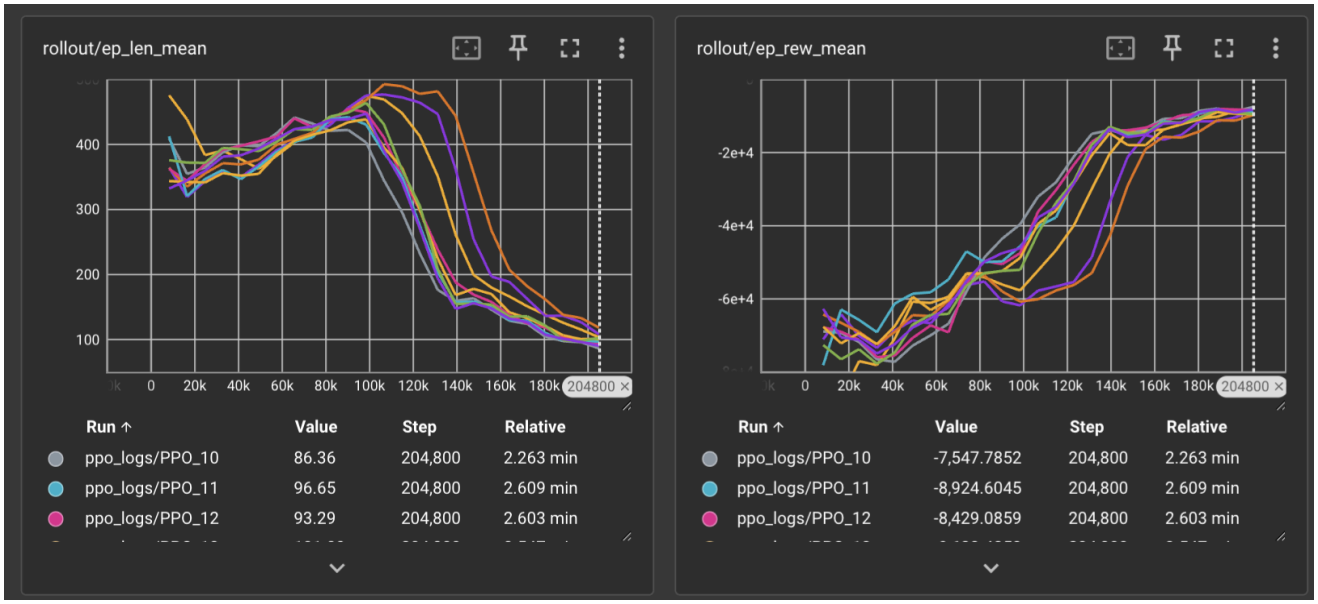
V6.7
Idea: [128, 128, 128]
Other noteworthy parameters: logs=PPO_10
Results: see tensorboard logs

V6.8
Idea: [128, 128, 128, 128]
Other noteworthy parameters: logs=PPO_11
Results: see tensorboard logs

V6.9
Idea: [256, 256]
Other noteworthy parameters: logs=PPO_12
Results: see tensorboard logs

V6.10
Idea: [256, 256, 256]
Other noteworthy parameters: logs=PPO_13
Results: see tensorboard logs

V6.11
Idea: [256, 256, 256, 256]
Other noteworthy parameters: logs=PPO_14
Results: see tensorboard logs



obviously look at the real tensorboard logs to see the full training curves. There is a difference between curves but most of the curves lie in the same area. The network with [128, 128] did the worst. The network with [128, 128, 128] performed the best. It seems like 128 is a nice middle ground for non overfitting but still having enough layers to have a good representation.

========== learning rate ==========

1e-5, 5e-5, 1e-4, 3e-4, 5e-4, 1e-3, 5e-3

V6.12

Idea: 1e-5

Other noteworthy parameters: logs=lr_exp/PPO_1

Results: see tensorboard logs

V6.13

Idea: 5e-5

Other noteworthy parameters: logs=lr_exp/PPO_2

Results: see tensorboard logs

V6.14

Idea: 1e-4

Other noteworthy parameters: logs=lr_exp/PPO_3

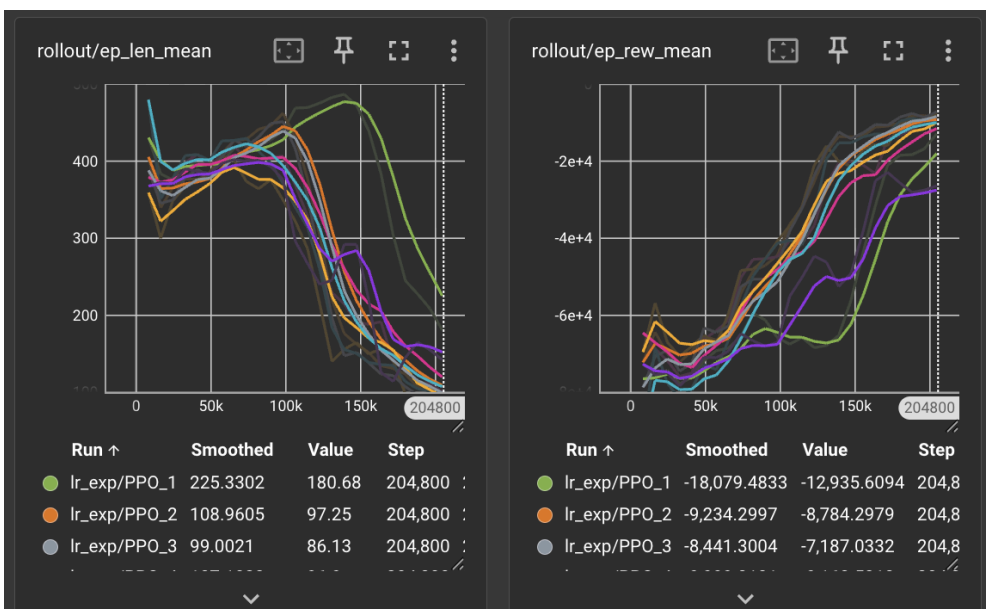
Results: see tensorboard logs

V6.15
Idea: 3e-4
Other noteworthy parameters: logs=lr_exp/PPO_4
Results: see tensorboard logs

V6.16
Idea: 5e-4
Other noteworthy parameters: logs=lr_exp/PPO_5
Results: see tensorboard logs

V6.17
Idea: 1e-3
Other noteworthy parameters: logs=lr_exp/PPO_6
Results: see tensorboard logs

V6.18
Idea: 5e-3
Other noteworthy parameters: logs=lr_exp/PPO_7
Results: see tensorboard logs

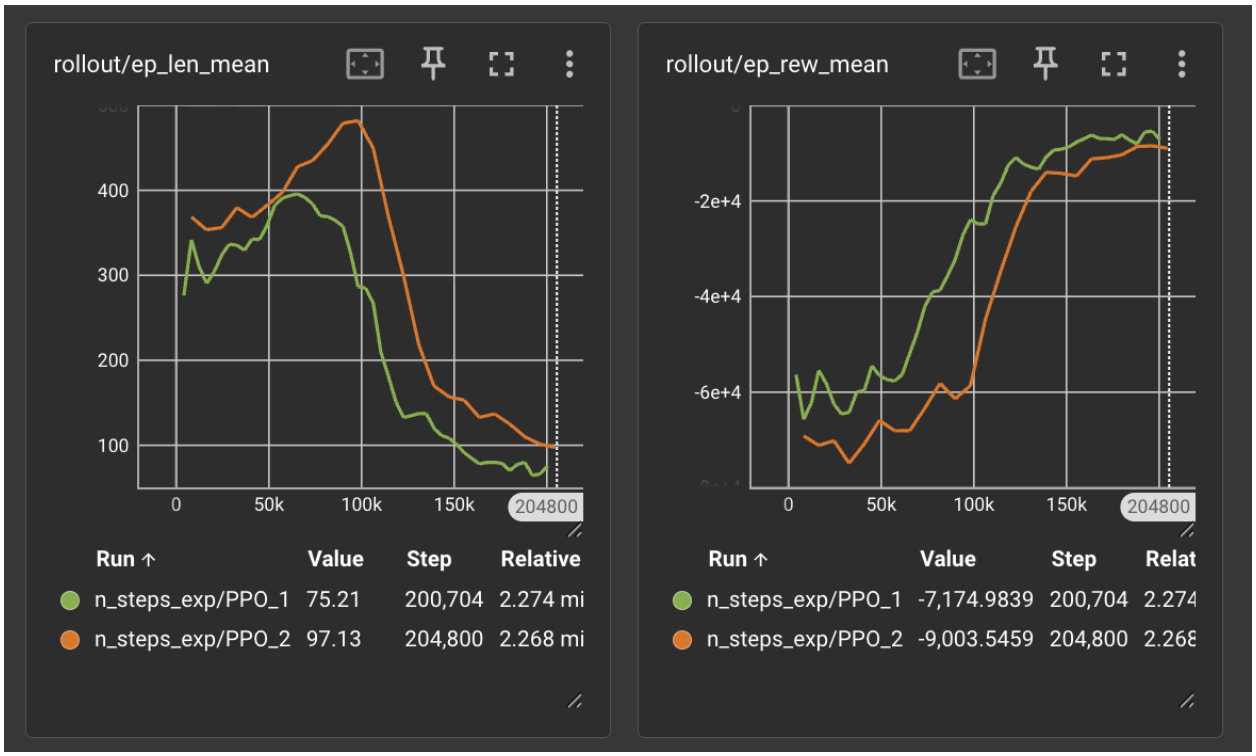


Increasing the learning rate only changes how fast the model learns. Too low, and the model takes too long to learn (green line). Too high, and the model makes too many changes and destabilizes training (purple line). Destabilization happened at a larger value than I thought (5e-3). I would feel safe implementing a learning rate of 3e-4, but I will bump it to 1e-4 for safety.

========== n_steps ==========
512 or 1024

V6.19
Idea: 512
Other noteworthy parameters: logs=n_steps_exp/PPO_1
Results: see tensorboard logs

V6.20
Idea: 1024
Other noteworthy parameters: logs=n_steps_exp/PPO_2
Results: see tensorboard logs



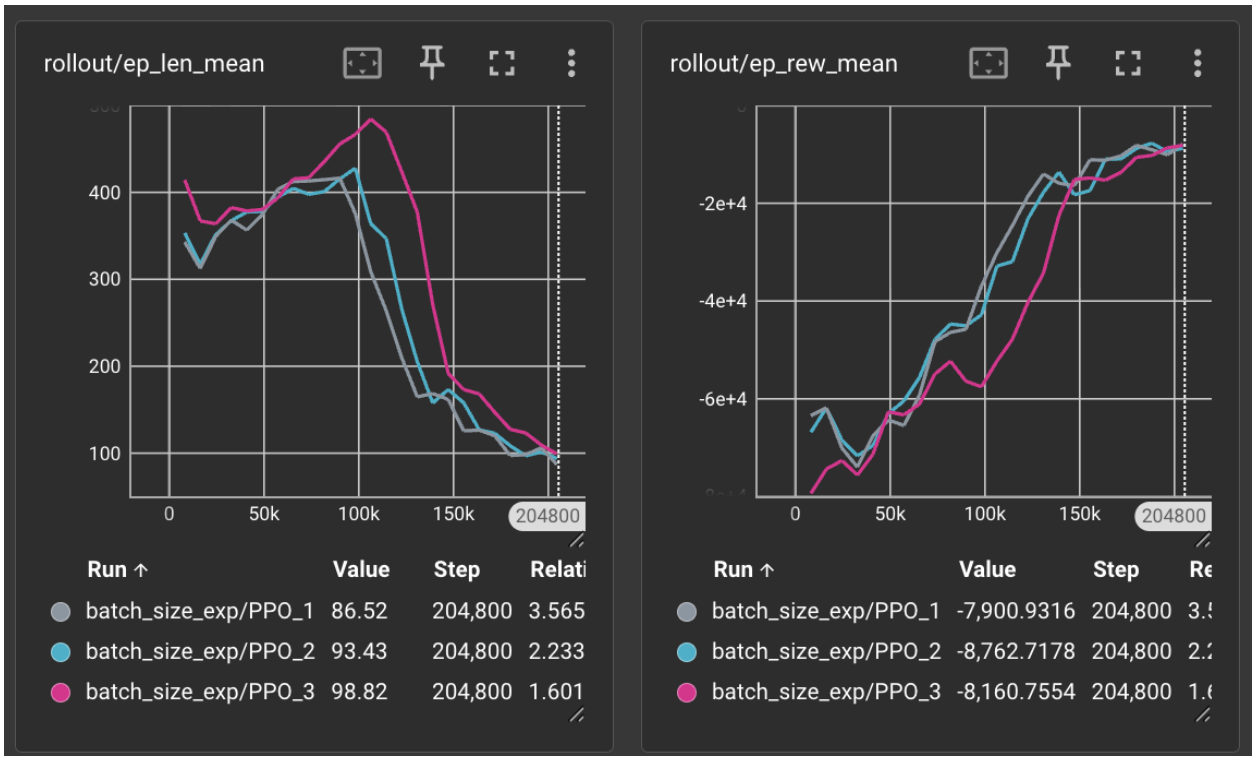
512 is better than 1024 in this very specific single set of hyperparameters for this very specific environment definition. but it's a better piece of evidence to choose one over the other than pure speculation, which is what i've been doing up until this point.

========== batch_size ==========
32, 64, 128

V6.21
Idea: 32
Other noteworthy parameters: logs=batch_size_exp/PPO_1
Results: see tensorboard logs

V6.22
Idea: 64
Other noteworthy parameters: logs=batch_size_exp/PPO_2
Results: see tensorboard logs

V6.23
Idea: 128
Other noteworthy parameters: logs=batch_size_exp/PPO_3
Results: see tensorboard logs



This is very surprising. It's also worth noting that the pink line (128) also took twice as long to finish running. For some reason (for this very specific experiment), batch size of 32 performs better.

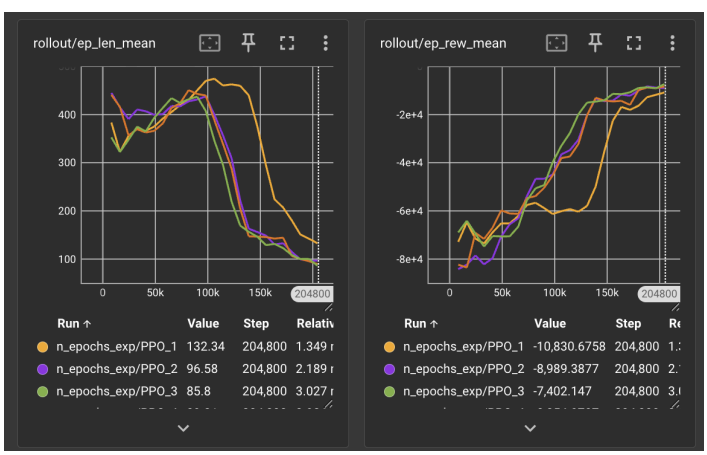
========== n_epochs ==========
5, 10, 15, 20

V6.24
Idea: 5
Other noteworthy parameters: logs=n_epochs_exp/PPO_1
Results: see tensorboard logs

V6.25
Idea: 10
Other noteworthy parameters: logs=n_epochs_exp/PPO_2
Results: see tensorboard logs

V6.26
Idea: 15
Other noteworthy parameters: logs=n_epochs_exp/PPO_3
Results: see tensorboard logs

V6.27
Idea: 20
Other noteworthy parameters: logs=n_epochs_exp/PPO_4
Results: see tensorboard logs



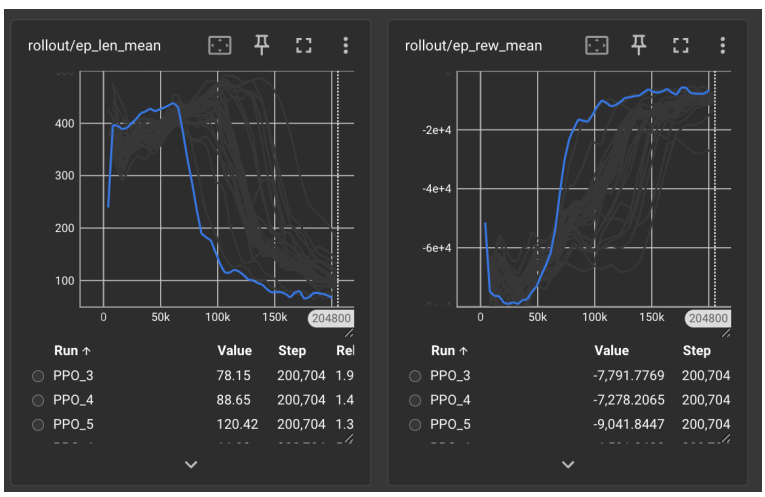
yellow (5) was the worst. green (15) was the best. although the difference between 10, 15, and 20 is very small.

========== results ==========

Taking all the best decisions from each experiment yields us with the following hyperparameters:

```
model = PPO(
    policy="MlpPolicy",
    env=env,
    device="cpu",
    policy_kwargs = dict(
        net_arch=dict(
            pi=[128, 128, 128],          # policy MLP
            vf=[128, 128, 128]           # value MLP
        ),
        activation_fn=nn.ReLU,
    ),
    learning_rate=1e-4,
    n_steps=512,
    batch_size=32,
    n_epochs=15,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.1,
    ent_coef=0.002,
    vf_coef=0.5,
    max_grad_norm=0.5,
    verbose=1,
    normalize_advantage=True,
    tensorboard_log="./ppo_logs/n_epochs_exp/",
)
```

V6.28 which trains as follows (logs: PPO_6):



The blue line is V6.28, and all the gray lines are every other model that was trained (V6.0-V6.27).

It seems like architecture plays the biggest role in how well the model trains. I'm going to run another experiment to fully explore architecture. The biggest thing I'll start with is varying the layer counts within networks. For example, instead of just training [128, 128, 128], I'll train things like [256, 128, 64, 64].

========== ARCHITECTURE EXPERIMENT 2 ==========
I will explore more layers (up to 4 now), but train into the same folder and use the new best hyperparameters (ppo_logs/architecture_exp). I will also vary 32->256 in powers of 2. If there is variation in layer sizes, they will always be decreasing.
[32, 32], [32, 32, 32], [32, 32, 32], [256, 256], [256, 256, 256], [256, 256, 256],
[256, 128], [256, 64], [128, 64],
[256, 256, 128], [256, 256, 64], [256, 128, 128], [256, 128, 64], [256, 64, 64],
[128, 128, 64], [128, 64, 64],
[256, 256, 256, 128], [256, 256, 128, 128], [256, 128, 128, 64], [256, 128, 64, 64],
[128, 128, 128, 64], [128, 128, 64, 64], [128, 64, 64, 64]

V6.29.1
Idea: [32, 32]
Other noteworthy parameters: logs=architecture_exp/PPO_15
Results: see tensorboard logs

V6.29.2
Idea: [32, 32, 32]
Other noteworthy parameters: logs=architecture_exp/PPO_16
Results: see tensorboard logs

V6.29.3
Idea: [32, 32, 32, 32]
Other noteworthy parameters: logs=architecture_exp/PPO_17
Results: see tensorboard logs

V6.29.4
Idea: [256, 256]
Other noteworthy parameters: logs=architecture_exp/PPO_18
Results: see tensorboard logs

V6.29.5
Idea: [256, 256, 256]
Other noteworthy parameters: logs=architecture_exp/PPO_19
Results: see tensorboard logs

V6.29.6
Idea: [256, 256, 256, 256]
Other noteworthy parameters: logs=architecture_exp/PPO_20
Results: see tensorboard logs

V6.29.7
Idea: [256, 128]
Other noteworthy parameters: logs=architecture_exp/PPO_21
Results: see tensorboard logs

V6.29.8
Idea: [256, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_22
Results: see tensorboard logs

V6.29.9
Idea: [128, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_23
Results: see tensorboard logs

V6.29.10
Idea: [256, 256, 128]
Other noteworthy parameters: logs=architecture_exp/PPO_24
Results: see tensorboard logs

V6.29.11
Idea: [256, 256, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_25
Results: see tensorboard logs

V6.29.12
Idea: [256, 128, 128]
Other noteworthy parameters: logs=architecture_exp/PPO_26
Results: see tensorboard logs

V6.29.13
Idea: [256, 128, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_27
Results: see tensorboard logs

V6.29.14
Idea: [256, 64, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_28
Results: see tensorboard logs

V6.29.15
Idea: [128, 128, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_29
Results: see tensorboard logs

V6.29.16
Idea: [128, 64, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_30
Results: see tensorboard logs

V6.29.17
Idea: [256, 256, 256, 128]
Other noteworthy parameters: logs=architecture_exp/PPO_31
Results: see tensorboard logs

V6.29.18
Idea: [256, 256, 128, 128]
Other noteworthy parameters: logs=architecture_exp/PPO_32
Results: see tensorboard logs

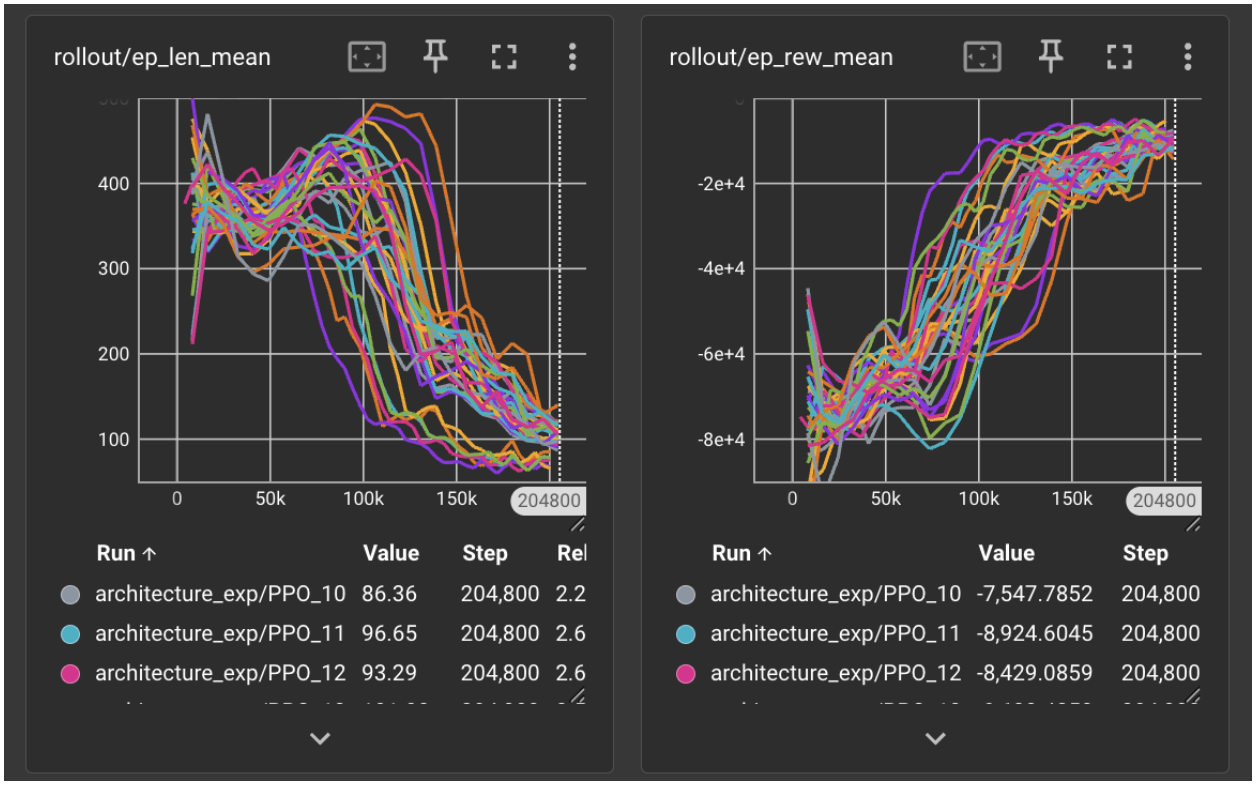
V6.29.19
Idea: [256, 128, 128, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_33
Results: see tensorboard logs

V6.29.20
Idea: [256, 128, 64, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_34
Results: see tensorboard logs

V6.29.21
Idea: [128, 128, 128, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_35
Results: see tensorboard logs

V6.29.22
Idea: [128, 128, 64, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_36
Results: see tensorboard logs

V6.29.23
Idea: [128, 64, 64, 64]
Other noteworthy parameters: logs=architecture_exp/PPO_37
Results: see tensorboard logs



This graph is basically nonsense: there is too much information. but all the plots are in the logs in the repository so any nicer graph can be made from the data. The important thing to take away from this is: architecture has a real effect on how well the network trains! AND SURPRISINGLY, the best agent had an architecture of [256, 256]. I want to explore this more.

========== ARCHITECTURE EXPERIMENT 3 ==========
The best results from the previous two experiments expanded on. We again modify the architecture.
[32], [64], [128], [256], [512], [1024]
[512, 512], [512, 256]

V6.30.1
Idea: [32]
Other noteworthy parameters: logs=architecture_exp/PPO_38
Results: see tensorboard logs

V6.30.2
Idea: [64]
Other noteworthy parameters: logs=architecture_exp/PPO_39
Results: see tensorboard logs

V6.30.3
Idea: [128]
Other noteworthy parameters: logs=architecture_exp/PPO_40
Results: see tensorboard logs

V6.30.4
Idea: [256]
Other noteworthy parameters: logs=architecture_exp/PPO_41
Results: see tensorboard logs

V6.30.5
Idea: [512]
Other noteworthy parameters: logs=architecture_exp/PPO_42
Results: see tensorboard logs

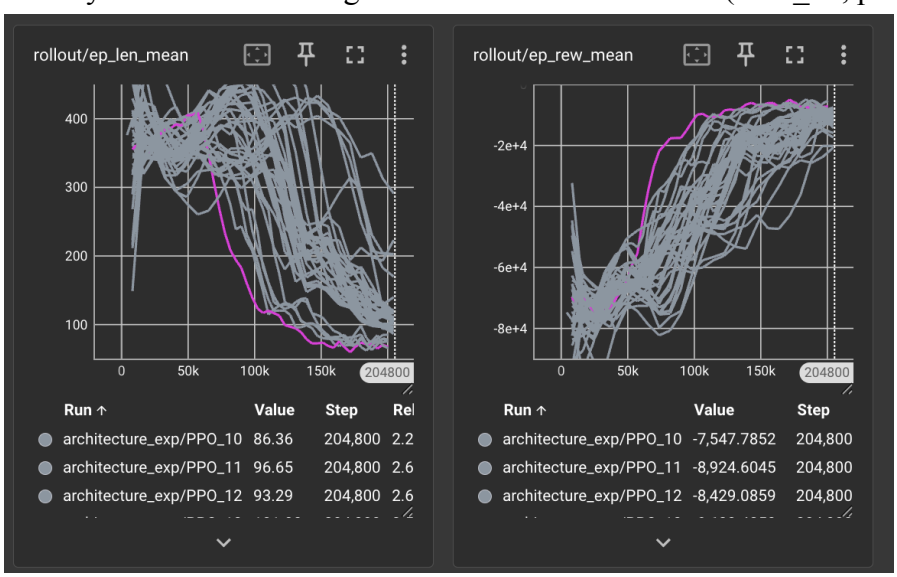
V6.30.6
Idea: [512, 512]
Other noteworthy parameters: logs=architecture_exp/PPO_43
Results: see tensorboard logs

V6.30.7
Idea: [512, 256]
Other noteworthy parameters: logs=architecture_exp/PPO_44
Results: see tensorboard logs

The best architecture is:

```python
model = PPO(
    policy="MlpPolicy",
    env=env,
    device="cpu",
    policy_kwargs = dict(
        net_arch=dict(
            pi=[256, 256],          # policy MLP
            vf=[256, 256]           # value MLP
        ),
        activation_fn=nn.ReLU,
    ),
    learning_rate=1e-4,
    n_steps=512,
    batch_size=32,
    n_epochs=15,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.1,
    ent_coef=0.002,
    vf_coef=0.5,
    max_grad_norm=0.5,
    verbose=1,
    normalize_advantage=True,
    tensorboard_log="./ppo_logs/architecture_exp/",
)
```

which yielded the following curves: best curve is 6.29.4 (PPO_18, pink line)



RESULTS:

The hyperparameters absolutely have an effect on the training curves. Mostly it's the architecture that has the most effect. I'm wary about what the best architecture will be once we change the environment. I think it will be different when we go to 2 panels. But these experiments provide a better basis for choosing hyperparameters than we had before.